



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de Software

Análisis y Diseño de Software - NRC 30723

U2T1-U2T2-U2T3

Patrones de diseño

Grupo

Egas Steven

Obando Alejandro

Quimbiulco Juan

Instructor: Ing. Jenny Alexandra Ruiz Robalino

Fecha: 03/06/2026

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: <u>Ciencias de la Computación</u>
	Carrera: <u>Ingeniería de Software</u> Asignatura: <u>Análisis y Diseño de Software</u> NRC: <u>30723</u>
	Apellidos y nombres del estudiante: <u>Egas Jácome Steven David</u> <u>Obando Zapata Alejandro Xavier</u> <u>Qumbiulco Carrion Juan Diego</u>

ANTECEDENTES

El presente taller forma parte de la secuencia académica de la Unidad 2. En la tarea U2T1 se diseñó e implementó la arquitectura de tres capas para el CRUD del estudiante con los atributos ID, Nombres y Edad, separando las responsabilidades en las capas de Datos, Lógica de Negocio y Presentación.

Posteriormente, en la tarea U2T2, se incorporaron los patrones estructurales Adapter y Decorator para mejorar la flexibilidad del sistema. Adapter permitió adaptar información proveniente de fuentes externas con formatos diferentes al modelo interno del estudiante, mientras que Decorator permitió agregar funcionalidades adicionales, como auditoría y validación, sin modificar las clases base del servicio CRUD.

Finalmente, en la tarea U2T3, se integraron los patrones de comportamiento Observer y Strategy. Observer se utilizó para gestionar eventos generados por las operaciones del CRUD, notificando automáticamente a componentes de auditoría e historial cuando un estudiante es creado, actualizado o eliminado. Por su parte, Strategy permitió seleccionar dinámicamente distintos algoritmos de búsqueda de estudiantes, como búsqueda por ID o búsqueda por Nombre, sin alterar la lógica principal de la aplicación.

La integración de la arquitectura de tres capas junto con los patrones Adapter, Decorator, Observer y Strategy permite obtener una solución más flexible, mantenible y escalable, manteniendo la trazabilidad con la ERS/SRS y aplicando principios de diseño orientado a objetos.

OBJETIVO

Diseñar e integrar una solución completa para el CRUD del estudiante basada en una arquitectura de tres capas y en la aplicación de los patrones de diseño Adapter, Decorator, Observer y Strategy, identificando las clases involucradas, sus responsabilidades, la interacción entre los patrones y su contribución a la flexibilidad, mantenibilidad y escalabilidad del sistema mediante diagramas, implementación y evidencias de funcionamiento.

DESARROLLO

U2T1. Diseño e implementación de la arquitectura de tres capas para el CRUD del estudiante

Propósito de la tarea

Diseñar e implementar una arquitectura de software en tres capas para el CRUD del estudiante, separando claramente la capa de Datos, la capa de Lógica_Negocio y la capa de Presentación. La solución debe permitir al menos las operaciones crear, consultar, actualizar y eliminar estudiantes con los campos ID, Nombres y Edad.

Código	Tipo de aprendizaje	Horas sugeridas	Dependencia lógica
U2T1	Autónomo	3 horas	ERS/SRS, requisitos funcionales del CRUD y modelo de análisis del estudiante

Insumos obligatorios

- ERS/SRS del proyecto con los requisitos funcionales del CRUD del estudiante.
- Entidad Estudiante con atributos ID, Nombres y Edad.
- Casos de uso o historias de usuario relacionadas con registrar, listar, modificar y eliminar estudiantes.
- Criterios de organización del proyecto en paquetes, carpetas, clases o componentes.

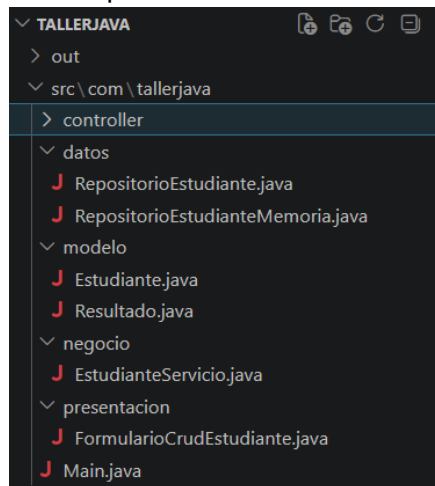
Desarrollo de la actividad

- Analizar la ERS/SRS y seleccionar los requisitos funcionales relacionados con el CRUD del estudiante.

- Identificar las clases base del proyecto, por ejemplo: *Estudiante*, *EstudianteDAO* o *RepositorioEstudiante*, *ServicioEstudiante* y *VistaEstudiante*.
- Diseñar la arquitectura en tres capas: Datos para persistencia, Lógica_Negocio para validaciones y reglas, y Presentación para interacción con el usuario.
- Implementar el CRUD base del estudiante utilizando los atributos ID, Nombres y Edad.
- Documentar la responsabilidad de cada capa y la relación entre requisitos, componentes y operaciones.
- Presentar evidencias de ejecución donde se observe al menos el registro y la consulta de estudiantes.

Productos verificables

- Proyecto base organizado en tres capas.



- Diagrama o esquema de arquitectura aplicado al CRUD del estudiante.

```
@startuml
    title Arquitectura en Tres Capas - CRUD de Estudiantes

    left to right direction

    skinparam shadowing false
    skinparam backgroundColor #FFFFFF
    skinparam defaultTextAlignment center
    skinparam rectangleRoundCorner 10
    skinparam componentStyle rectangle
    skinparam packageStyle rectangle
    skinparam ArrowColor #555555
    skinparam ArrowThickness 1.2
    skinparam linetype ortho

    skinparam rectangle {
        BorderColor #4A5568
        FontColor #1A202C
        FontSize 13
    }

    skinparam package {
        BorderColor #4A5568
        FontColor #1A202C
        FontSize 15
        FontStyle bold
    }

    actor "Usuario" as Usuario #FFFFFF

    package "Capa de Presentación" as Presentacion #DCEEFF {
        rectangle "FormularioCrudEstudiante\n<<Swing JFrame>>" as Formulario #F7FBFF
    }

    package "Capa de Lógica de Negocio" as Negocio #E1F5E6 {
```

```

rectangle "EstudianteServicio\n<<Servicio CRUD>>" as Servicio #F8FFF9
rectangle "Resultado\n<<Respuesta de operación>>" as Resultado #F8FFF9
}

package "Capa de Datos" as Datos #F1E3FA {
  rectangle "RepositorioEstudiante\n<<Interface>>" as RepoInterface #FCF7FF
  rectangle "RepositorioEstudianteMemoria\n<<Implementación>>" as RepoMemoria #FCF7FF
  database "Colección en memoria" as Memoria #FFFFFF
}

package "Modelo" as Modelo #FFF4CC {
  rectangle "Estudiante\nid : String\nnombre : String\nedad : int" as Estudiante #FFFDF2
}

rectangle "Main\n<<Punto de entrada>>" as Main #EEEEEE

Usuario --> Formulario : usa

Main --> RepoMemoria : instancia
Main --> Servicio : instancia
Main --> Formulario : inicia

Formulario --> Servicio : solicitudes CRUD

Servicio --> Resultado : genera
Servicio --> Estudiante : crea / valida
Servicio --> RepoInterface : operaciones CRUD

RepoInterface <|.. RepoMemoria
RepoMemoria --> Memoria : guarda / consulta
RepoMemoria --> Estudiante : administra

note top of Presentacion #F9FAFB
Interfaz gráfica que captura datos
y muestra la tabla de estudiantes.
end note

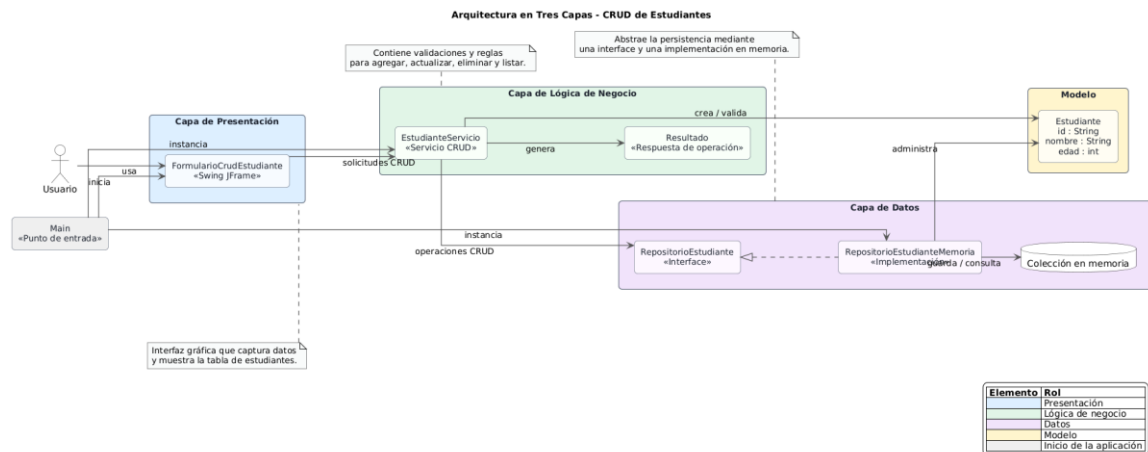
note top of Negocio #F9FAFB
Contiene validaciones y reglas
para agregar, actualizar, eliminar y listar.
end note

note bottom of Datos #F9FAFB
Abstrae la persistencia mediante
una interface y una implementación en memoria.
end note

legend right
|= Elemento |= Rol |
|<#DCEEFF>| Presentación |
|<#E1F5E6>| Lógica de negocio |
|<#F1E3FA>| Datos |
|<#FFF4CC>| Modelo |

|<#EEEEEE>| Inicio de la aplicación |
endlegend

```



- Tabla de responsabilidades por capa.

Capa	Componentes	Responsabilidad
Presentación	FormularioCrudEstudiante	Permite la interacción con el usuario mediante una interfaz gráfica. Captura los datos del estudiante (ID, Nombres y Edad), envía las solicitudes a la lógica de negocio y muestra los resultados obtenidos.
Lógica de Negocio	EstudianteServicio	Implementa las reglas de negocio del sistema. Valida los datos ingresados, verifica restricciones como ID único y edad válida, y coordina las operaciones CRUD antes de comunicarse con la capa de datos.
Datos	RepositorioEstudiante, RepositorioEstudianteMemoria	Gestiona el almacenamiento y recuperación de los estudiantes. Proporciona las operaciones de guardar, buscar, actualizar, eliminar y listar registros.

- Matriz de trazabilidad entre requisitos del CRUD y componentes implementados.

Requisito	Operación CRUD	Componente de presentación	Componente de lógica de negocio	Componente de datos	Evidencia esperada
RF-01: Agregar estudiante	Crear	FormularioCrudEstudiante	EstudianteServicio	RepositorioEstudianteMemoria	El estudiante se registra con ID, Nombres y Edad.
RF-02: Actualizar estudiante	Actualizar	FormularioCrudEstudiante	EstudianteServicio	RepositorioEstudianteMemoria	Los datos del estudiante seleccionado se modifican correctamente.
RF-03: Eliminar estudiante	Eliminar	FormularioCrudEstudiante	EstudianteServicio	RepositorioEstudianteMemoria	El estudiante seleccionado se elimina del listado.
RF-04: Mostrar estudiantes	Consultar/Listar	FormularioCrudEstudiante	EstudianteServicio	RepositorioEstudianteMemoria	Se muestra una tabla con todos los estudiantes registrados.
RF-05: Validar datos del estudiante	Validación previa a Crear y Actualizar	FormularioCrudEstudiante	EstudianteServicio	No aplica directamente	El sistema impide guardar datos vacíos, edad inválida o ID duplicado.

Capturas o pruebas de ejecución del registro, consulta, actualización y eliminación de estudiantes.

Sistema CRUD de Estudiantes

Datos del Estudiante

ID:

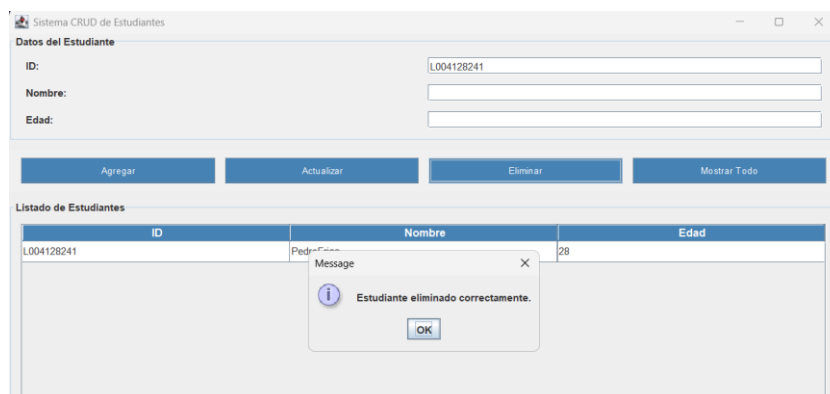
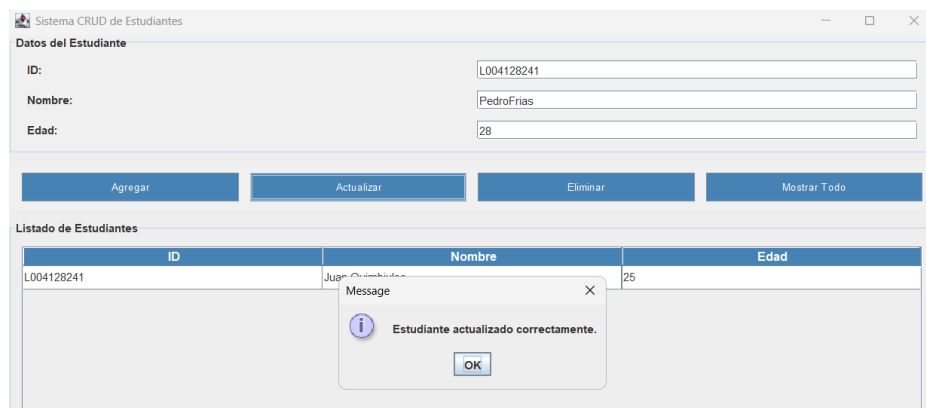
Nombre:

Edad:

Listado de Estudiantes

ID	Nombre	Edad

Message: Estudiante agregado correctamente.



U2T2

1. Patrón Adapter

1.1 ¿Qué es el patrón Adapter?

Es un patrón de diseño estructural que permite que dos interfaces incompatibles trabajen juntas. Actúa como un intermediario que convierte la interfaz de una clase en la interfaz que el cliente espera, sin modificar ni la clase original ni el cliente. Es como un adaptador de enchufe: no cambia el dispositivo ni el tomacorriente, solo hace que sean compatibles.

1.2 Características

- Convierte una interfaz incompatible en la interfaz esperada por el cliente.
- No modifica ni la clase original (Adaptee) ni el cliente.
- Permite reutilizar clases existentes aunque su interfaz no sea compatible.
- Puede implementarse por composición (Adapter de objeto) o por herencia múltiple.

1.3 ¿Por qué usar Adapter en el CRUD del estudiante?

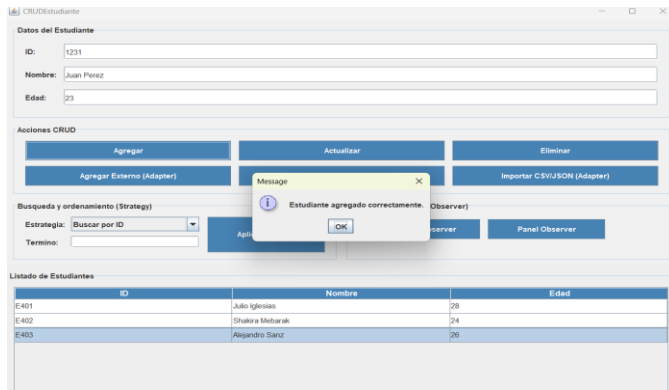
El sistema interno usa los campos ID, Nombres y Edad. Sin embargo, puede llegar información desde fuentes externas (CSV, JSON o API) con campos distintos: codigo, nombreCompleto y anios. Sin Adapter, habría que modificar el repositorio o el servicio para aceptar ese formato, rompiendo la arquitectura. Con Adapter, la clase AdaptadorEntradaEstudiante convierte ese formato externo al objeto Estudiante que el sistema ya conoce, sin alterar ninguna clase existente.

1.4 Mapeo del patrón Adapter en el proyecto

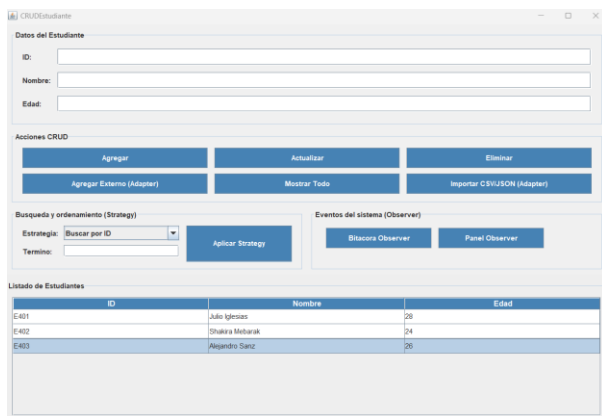
Elemento del patrón Adapter	Clase en el proyecto	Responsabilidad
Target (interfaz esperada)	ServicioCRUDEstudiante	Interfaz que define las operaciones del CRUD que el cliente espera: crearEstudiante(id, nombres, edad).
Adaptee (clase incompatible)	Datos externos (CSV / JSON / API)	Fuente de datos externa con campos incompatibles: codigo, nombreCompleto, anios.
Adapter	AdaptadorEntradaEstudiante	Convierte los datos externos al formato Estudiante(ID, Nombres, Edad) y delega al ServicioCRUDEstudiante.
Client	VistaEstudiante	Usa el Adapter para importar estudiantes externos sin conocer el formato de origen.

2.4 Funcionalidad del patrón Adapter en el proyecto

Antes



Después



2. Patrón Decorator

2.1 ¿Qué es el patrón Decorator?

Es un patrón de diseño estructural que permite agregar nuevas responsabilidades a un objeto de forma dinámica, envolviéndolo con una clase que añade el nuevo comportamiento antes o después de delegar al objeto base, sin alterar su clase original. Es como envolver un regalo: el contenido no cambia, pero se le agrega algo por fuera.

2.2 Características

- Extiende el comportamiento de un objeto sin modificar su clase original.
- Aplica el principio abierto/cerrado: abierto para extensión, cerrado para modificación.
- Se pueden apilar múltiples decoradores sobre el mismo objeto.
- Usa composición en vez de herencia para agregar funcionalidad.

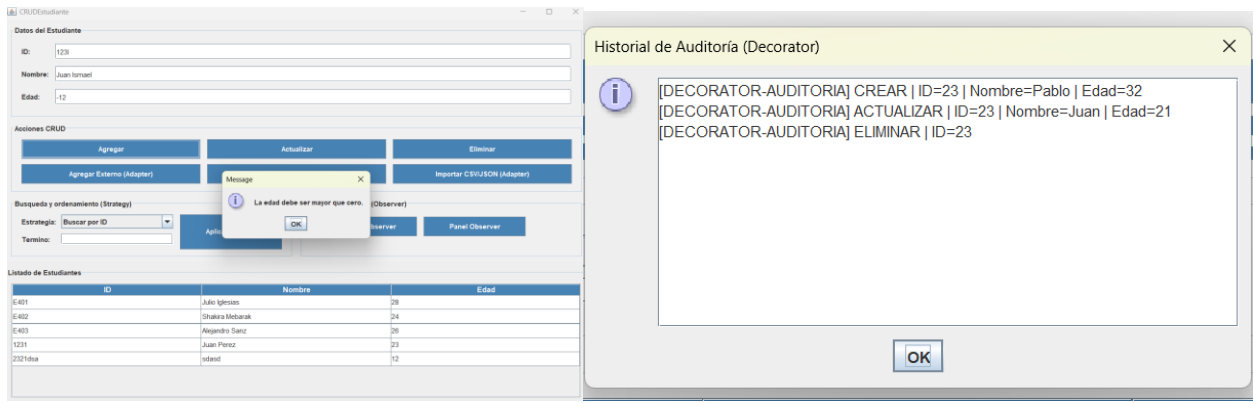
2.3 ¿Por qué usar Decorator en el CRUD del estudiante?

El ServicioCRUDEstudiante ya implementa correctamente las operaciones base. Sin embargo, se necesita registrar auditoría y aplicar validaciones adicionales cada vez que se crea o actualiza un estudiante. Modificar directamente el servicio base mezclaría responsabilidades. Con Decorator, la clase AdaptadorAuditoriaValidacion envuelve al servicio base y agrega ese comportamiento antes o después de cada operación, sin tocar el código original.

2.4 Mapeo del patrón Decorator en el proyecto

Elemento del patrón Decorator	Clase en el proyecto	Responsabilidad
Component (interfaz)	ServicioCRUDEstudiante (interfaz)	Interfaz común que define las operaciones del CRUD. Tanto el servicio base como el decorador la implementan.
Concrete Component	ServicioCRUDEstudiante (implementación)	Implementación original del CRUD. Contiene la lógica base sin comportamientos adicionales.
Concrete Decorator	AdaptadorAuditoriaValidacion	Envuelve al servicio base. Agrega auditoría y validación al crear y actualizar estudiantes, sin modificar el servicio original.
Client	VistaEstudiante	Trabaja con la interfaz del servicio sin saber si está usando el servicio base o el decorado.

2.4 Funcionalidad del patrón Decorator en el proyecto



3. Diagrama de arquitectura integrado – Adapter y Decorator aplicados

El siguiente diagrama muestra la arquitectura completa del CRUD del estudiante con los dos patrones integrados en las tres capas del sistema:

```
@startuml
    U2T2_Arquitectura_Adapter_Decorator
    title Arquitectura integrada del CRUD del estudiante - U2T2\nTres capas + Adapter + Decorator
```

```
left to right direction
skinparam shadowing false
skinparam defaultTextAlignment center
skinparam rectangleRoundCorner 12
skinparam componentStyle rectangle
skinparam packageStyle rectangle
```

```
actor "Estudiante /\nUsuario" as Usuario
```

```
cloud "Datos externos\nCSV / JSON / API" as Externo #FFF2CC
```

```
rectangle "Capa de Presentación" as CAPA_PRESENTACION #D9EAF7 {
    rectangle "VistaEstudiante\nFormulario / Tabla CRUD" as Vista #EAF4FB
}
```

```
rectangle "Capa de Lógica Negocio" as CAPA_NEGOCIO #DFF2E1 {
    rectangle "AdaptadorAuditoriaValidacion\n(Decorator)" as Decorator #FFE6CC
    rectangle "AdaptadorEntradaEstudiante\n(Adapter)" as Adapter #FCE5CD
    rectangle "ServicioCRUDEstudiante\n(Servicio base)" as Servicio #F3FBF4
}
```

```
rectangle "Capa de Datos" as CAPA_DATOS #F3E0F7 {
    rectangle "RepositorioEstudiante" as Repositorio #FAF1FC

    database "Fuente de datos interna\nMemoria / Archivo / BD" as FuenteDatos #FFFFFF

    rectangle "Estudiante\n(ID, Nombres, Edad)" as Entidad #FCF7FD
}
```

```
Usuario --> Vista : interacción
```

```
Vista --> Decorator : solicitudes CRUD
```

```
Decorator --> Servicio : agrega auditoría\ny validación
```

```
Externo --> Adapter : codigo,\nnombreCompleto,\nanios
```

```
Adapter --> Servicio : convierte formato\nexterno a Estudiante
```

```
Servicio --> Repositorio : guardar /\nlistar /\nactualizar /\neliminar
```

```
Repositorio --> FuenteDatos : persistencia
```

```
Repositorio --> Entidad : almacena /\nrecupera
```

```
note bottom of Adapter #FFF2CC
Adapter:
```



```

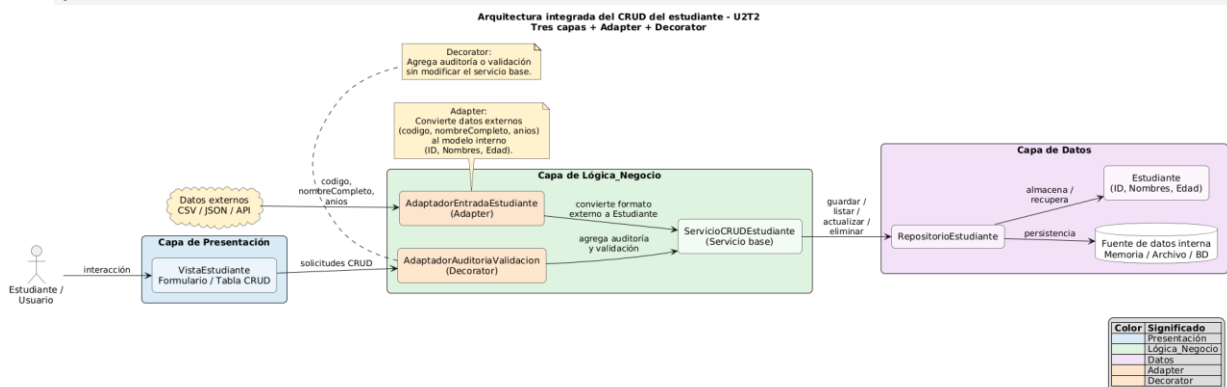
Convierte datos externos
(codigo, nombreCompleto, anios)
al modelo interno
(ID, Nombres, Edad).
end note

note bottom of Decorator #FFF2CC
Decorator:
Agrega auditoría o validación
sin modificar el servicio base.
end note

legend right
|= Color |= Significado |
|<#D9EAF7>| Presentación |
|<#DFF2E1>| Lógica_Negocio |
|<#F3E0F7>| Datos |
|<#FCE5CD>| Adapter |
|<#FFE6CC>| Decorator |
endlegend

@enduml

```



4. Integración de Adapter y Decorator con la arquitectura de tres capas

Capa	Componente	Rol en la arquitectura
Presentación	VistaEstudiante (Formulario / Tabla CRUD)	Recibe la interacción del usuario. Envía solicitudes al Decorator y usa el Adapter para importar datos externos.
Lógica Negocio	AdaptadorAuditoriaValidacion (Decorator)	Envuelve al servicio base. Agrega auditoría y validación sin modificar ServicioCRUDEstudiante.
Lógica Negocio	AdaptadorEntradaEstudiante (Adapter)	Recibe datos externos (codigo, nombreCompleto, anios) y los convierte al modelo interno Estudiante(ID, Nombres, Edad).
Lógica Negocio	ServicioCRUDEstudiante (Servicio base)	Implementa las operaciones CRUD puras. No fue modificado al aplicar ninguno de los dos patrones.
Datos	RepositorioEstudiante	Gestiona la persistencia. Almacena y recupera objetos Estudiante(ID, Nombres, Edad).
Datos	Fuente de datos interna (Memoria / Archivo / BD)	Almacenamiento fisico de los datos. No fue modificado al aplicar los patrones.

U2T3

1. Patrón Observer

1.1 ¿Qué es el patrón Observer?

Es un patrón de diseño de comportamiento que define una dependencia uno-a-muchos entre objetos, de modo que cuando un objeto (sujeto) cambia de estado, todos sus dependientes (observadores) son notificados y actualizados automáticamente. Permite implementar sistemas de manejo de eventos donde el sujeto no conoce los detalles de sus observadores.

1.2 Características

- Establece una relación uno-a-muchos sin acoplamiento directo entre sujeto y observadores.
- El sujeto solo conoce la interfaz Observer, no las clases concretas.

- Los observadores pueden ser agregados o removidos en tiempo de ejecución.

1.3 ¿Por qué usar Observer en el CRUD del estudiante?

El GestorEstudiante (Subject) realiza operaciones CRUD que generan eventos relevantes para el sistema: creación, actualización y eliminación de estudiantes. Sin Observer, habría que modificar el servicio cada vez que se necesite una nueva reacción a un evento (registrar en bitácora, notificar en pantalla, enviar correo, etc.), mezclando responsabilidades y violando el principio de responsabilidad única. Con Observer, el gestor simplemente notifica a su lista de observadores tras cada operación, y cada observador implementa su propia respuesta sin alterar el sujeto.

1.4 Mapeo del patrón Observer en el proyecto

<u>Elemento del patrón Observer</u>	<u>Clase en el proyecto</u>	<u>Responsabilidad</u>
Subject (Sujeto observable)	GestorEstudiante	Mantiene la lista de observadores registrados y notifica a todos cuando ocurre un evento CRUD (crear, actualizar, eliminar).
Observer (interfaz)	ObservadorEstudiante	Define el contrato <code>update(evento: String, estudiante: Estudiante)</code> que todos los observadores concretos deben implementar.
ConcreteObserver 1	ObservadorBitacora	Registra en memoria un historial detallado de cada evento: operación, ID, nombre, edad y marca de tiempo.
ConcreteObserver 2	ObservadorConsola	Imprime en consola un mensaje inmediato cada vez que ocurre un evento, útil para depuración y monitoreo en tiempo real.
ConcreteObserver 3	ObservadorPanelNotificaciones	Acumula mensajes cortos para mostrar en el panel de notificaciones de la interfaz gráfica Swing.
Client	FormularioCrudEstudiante	Crea el GestorEstudiante, registra los observadores y delega las operaciones CRUD al gestor, que dispara las notificaciones automáticamente.

2. Patrón Strategy

2.1 ¿Qué es el patrón Strategy?

Es un patrón de diseño de comportamiento que define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables. Strategy permite que el algoritmo varíe independientemente de los clientes que lo usan. En lugar de implementar múltiples variantes de un comportamiento en una sola clase mediante condicionales, cada variante se encapsula en su propia clase estrategia concreta.

2.2 Características

- Encapsula algoritmos en clases separadas con una interfaz común.
- Permite cambiar el algoritmo en tiempo de ejecución sin modificar el cliente.
- Elimina condicionales (if/switch) en el contexto para seleccionar comportamientos.

2.3 ¿Por qué usar Strategy en el CRUD del estudiante?

El sistema requiere ofrecer al usuario distintas formas de consultar el listado de estudiantes: buscar por ID exacto, buscar por nombre (parcial), ordenar por edad y ordenar por ID. Sin Strategy, el FormularioCrudEstudiante necesitaría un bloque if/else o switch para seleccionar el algoritmo, acoplando la presentación a la lógica de búsqueda y dificultando agregar nuevas formas de consulta. Con Strategy, el GestorEstudiante actúa como Contexto y delega la operación a la estrategia activa, que puede cambiarse dinámicamente desde la interfaz sin tocar ni el gestor ni el formulario.

2.4 Mapeo del patrón Strategy en el proyecto

<u>Elemento del patrón Strategy</u>	<u>Clase en el proyecto</u>	<u>Responsabilidad</u>
Strategy (interfaz)	EstrategiaBusqueda	Define el contrato <code>buscar(termino: String, lista: List<Estudiante>): List<Estudiante></code> que todas las estrategias concretas deben cumplir.
ConcreteStrategy 1	BusquedaPorId	Filtra la lista comparando el término exacto (ignorando mayúsculas) con el ID de cada estudiante.
ConcreteStrategy 2	BusquedaPorNombre	Filtra la lista comprobando si el nombre del estudiante contiene el término de búsqueda (búsqueda parcial, insensible a mayúsculas).

ConcreteStrategy 3	<i>OrdenamientoPorEdad</i>	<i>Ordena la lista de estudiantes de menor a mayor edad sin aplicar filtro, útil para visualizar rangos etarios.</i>
ConcreteStrategy 4	<i>OrdenamientoPorId</i>	<i>Ordena la lista de estudiantes alfabéticamente por ID, facilitando la navegación ordenada del listado.</i>
Context	<i>GestorEstudiante</i>	<i>Contiene la estrategia activa y expone buscarConEstrategia(). Permite cambiarla en tiempo de ejecución con setEstrategia() sin modificar el cliente.</i>
Client	<i>FormularioCrudEstudiante</i>	<i>Selecciona la estrategia activa mediante un JComboBox y solicita la búsqueda/ordenamiento al contexto, sin conocer los detalles del algoritmo.</i>

3. Diagrama de arquitectura integrado – Observer, Strategy, Decorator y Adapter aplicados

```
@startuml
    title Arquitectura integrada del CRUD Estudiante
    Tres capas + Adapter + Decorator + Observer + Strategy
    left to right direction
    skinparam shadowing false
    skinparam defaultTextAlignment center
    skinparam rectangleRoundCorner 12
    skinparam componentStyle rectangle
    skinparam packageStyle rectangle

    actor "Estudiante /\nUsuario" as Usuario

    cloud "Datos externos\nCSV / JSON / API" as Externo #FFF2CC

    rectangle "Capa de Presentacion" as CAPA_PRESENTACION #D9EAF7 {
        rectangle "VistaCRUDEstudiante\nFormulario / Tabla CRUD" as Vista #EAF4FB
    }

    rectangle "Capa de Logica_Negocio" as CAPA_NEGOCIO #DFF2E1 {
        rectangle "AdaptadorEntradaEstudiante\n(Adapter)" as Adapter #FCE5CD

        rectangle "ServicioCRUDEstudiante\n(Servicio base)" as Servicio #F3FBF4

        rectangle "ValidacionAuditoriaDecorator\n(Decorator)" as Decorator #FFE6CC

        rectangle "GestorCRUDEstudiante\n(Subject + Context)" as Gestor #EADCF8

        rectangle "BitacoraObserver" as ObsBitacora #F4D9E7
        rectangle "ConsolaObserver" as ObsConsola #F4D9E7
        rectangle "PanelNotificacionesObserver" as ObsPanel #F4D9E7

        rectangle "ContextoBusqueda\n(Strategy Context)" as CtxBusqueda #DCEBFA
        rectangle "BuscarPorID" as EstrategiaID #E8F4FD
        rectangle "BuscarPorNombre" as EstrategiaNombre #E8F4FD
        rectangle "OrdenarPorEdad" as EstrategiaEdad #E8F4FD
        rectangle "OrdenarPorID" as EstrategiaOrdenID #E8F4FD
    }

    rectangle "Capa de Datos" as CAPA_DATOS #F3E0F7 {
        rectangle "RepositorioEstudiante" as Repositorio #FAF1FC
        database "Fuente de datos interna\nMemoria" as FuenteDatos #FFFFFF
        rectangle "Estudiante\n(ID, Nombres, Edad)" as Entidad #FCF7FD
    }

    Usuario --> Vista : interaccion

    Vista --> Adapter : datos externos
    Externo --> Adapter : codigo,\nnombreCompleto,\nanios
    Adapter --> Gestor : Estudiante adaptado

    Vista --> Gestor : solicitudes CRUD
    Gestor --> Decorator : delega operaciones

    Decorator --> Servicio : valida y audita
    Servicio --> Repositorio : guardar / listar /\nactualizar / eliminar

    Repositorio --> FuenteDatos : persistencia
    Repositorio --> Entidad : almacena / recupera

    Gestor --> ObsBitacora : evento CRUD
```

```

Gestor --> ObsConsola : evento CRUD
Gestor --> ObsPanel : evento CRUD

Vista --> CtxBusqueda : selecciona estrategia
CtxBusqueda --> EstrategiaID : buscar por ID
CtxBusqueda --> EstrategiaNombre : buscar por nombre
CtxBusqueda --> EstrategiaEdad : ordenar por edad
CtxBusqueda --> EstrategiaOrdenID : ordenar por ID
CtxBusqueda --> Gestor : ejecutar busqueda
Gestor --> Repositorio : obtener listado

```

```

note top of CtxBusqueda #FFF2CC
Strategy:
Permite cambiar dinamicamente
entre busqueda por ID,
busqueda por nombre,
ordenamiento por edad
u ordenamiento por ID.
end note

```

```

note top of Gestor #FFF2CC
Observer:
GestorCRUDEstudiante notifica
eventos cuando se crea,
actualiza o elimina
un estudiante.
end note

```

```

note bottom of Decorator #FFF2CC
Decorator:
Agrega validacion y auditoria
sin modificar el servicio base.
end note

```

```

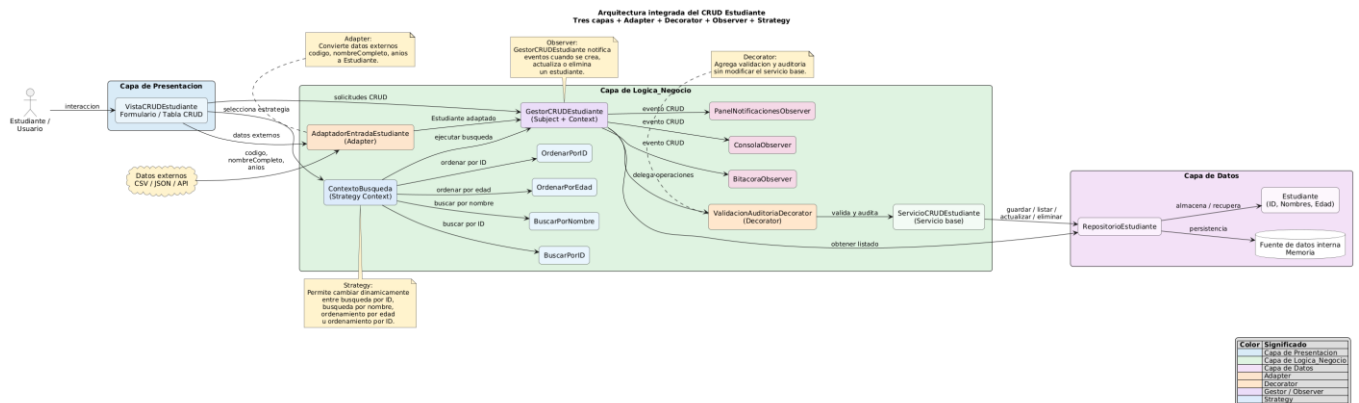
note bottom of Adapter #FFF2CC
Adapter:
Convierte datos externos
codigo, nombreCompleto, anios
a Estudiante.
end note

```

```

legend right
|= Color |= Significado |
|<#D9EAF7>| Capa de Presentacion |
|<#DFF2E1>| Capa de Logica Negocio |
|<#F3E0F7>| Capa de Datos |
|<#FCE5CD>| Adapter |
|<#FFE6CC>| Decorator |
|<#EADCF8>| Gestor / Observer |
|<#DCEBFA>| Strategy |
endlegend
@enduml

```



3. Integración de Observer y Strategy con la arquitectura de tres capas

Capa	Componente	Patrón integrado
------	------------	------------------

Presentación	FormularioCrudEstudiante	Client de Observer y Strategy. Registra observadores en el gestor y selecciona la estrategia de búsqueda activa desde un JComboBox.
Lógica de Negocio	GestorEstudiante	Subject de Observer y Context de Strategy. Centraliza las operaciones CRUD, notifica a observadores y ejecuta la estrategia activa.
Lógica de Negocio	ObservadorBitacora / ObservadorConsola / ObservadorPanelNotificaciones	ConcreteObservers. Reaccionan a eventos CRUD sin que el gestor conozca su implementación interna.
Lógica de Negocio	BusquedaPorId / BusquedaPorNombre / OrdenamientoPorEdad / OrdenamientoPorId	ConcreteStrategies. Encapsulan algoritmos de búsqueda y ordenamiento, intercambiables en tiempo de ejecución.
Datos	RepositorioEstudianteMemoria	No fue modificado. Los patrones de comportamiento actúan en capas superiores sin tocar la persistencia.

4. Matriz de trazabilidad: requisitos, eventos, estrategias y componentes

RF-01: Registrar estudiante	<i>estudianteCreado</i>	<i>ObservadorBitacora, ObservadorConsola, ObservadorPanelNotificaciones</i>	<i>GestorEstudiante → notificar() → ObservadorBitacora / ObservadorConsola / ObservadorPanelNotificaciones</i>
RF-02: Actualizar estudiante	<i>estudianteActualizado</i>	<i>ObservadorBitacora, ObservadorConsola, ObservadorPanelNotificaciones</i>	<i>GestorEstudiante → notificar() → ObservadorBitacora / ObservadorConsola / ObservadorPanelNotificaciones</i>
RF-03: Eliminar estudiante	<i>estudianteEliminado</i>	<i>ObservadorBitacora, ObservadorConsola, ObservadorPanelNotificaciones</i>	<i>GestorEstudiante → notificar() → ObservadorBitacora / ObservadorConsola / ObservadorPanelNotificaciones</i>
RF-04: Buscar por ID	<i>buscarConEstrategia(término)</i>	<i>BusquedaPorId</i>	<i>GestorEstudiante.setEstrategia(new BusquedaPorId()) → buscarConEstrategia(id)</i>
RF-05: Buscar por Nombre	<i>buscarConEstrategia(término)</i>	<i>BusquedaPorNombre</i>	<i>GestorEstudiante.setEstrategia(new BusquedaPorNombre()) → buscarConEstrategia(nombre)</i>

5. Evidencias de ejecución – Pruebas básicas

5.1 Código de prueba

Strategy

Busqueda y ordenamiento (Strategy)

Estrategia: Buscar por ID

Termino:

Aplicar Strategy

Eventos del sistema (Observer)

Bitacora Observer

Panel Observer

Listado de Estudiantes

ID	Nombre	Edad
1	Alejandro	22

Busqueda y ordenamiento (Strategy)

Estrategia: Ordenar por Edad

Termino:

Aplicar Strategy

Eventos del sistema (Observer)

Bitacora Observer

Panel Observer

Listado de Estudiantes

ID	Nombre	Edad
2	Steven	21
1	Alejandro	22
3	Juan	23

Observer

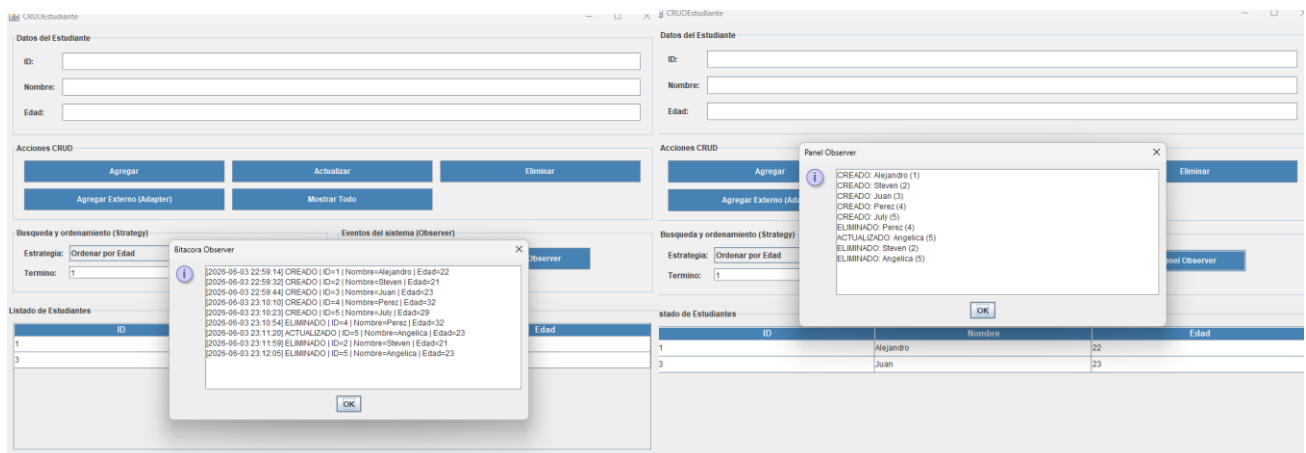


Tabla de Prueba del Observer y Strategy

ID	Patrón	Componente / Flujo	Escenario de Prueba	Resultado Esperado en Interfaz
TC-01	Strategy	ComboBox de Estrategia	Seleccionar la estrategia 'Ordenar por Edad' en el menú desplegable y presionar 'Aplicar Strategy'.	El componente 'Listado de Estudiantes' se reordena de forma ascendente según las edades (ej. Steven [21], Alejandro [22], Juan [23]).
TC-02	Strategy	Filtrado con Término	Cambiar la estrategia a una basada en búsqueda e ingresar un valor numérico o de texto en el campo 'Termino'.	La tabla se reduce dinámicamente mostrando únicamente las filas que coinciden plenamente con el criterio de filtrado.
TC-03	Observer	Notificación de Creación	Rellenar Datos del Estudiante (ID, Nombre, Edad) y presionar el botón 'Agregar'.	1. Panel Observer añade la línea: CREADO: [Nombre] ([ID]). 2. Bitácora Observer registra la traza completa con fecha, hora y todos los atributos.
TC-04	Observer	Notificación de Actualización	Seleccionar un estudiante existente en la tabla, modificar sus datos en el formulario y presionar 'Actualizar'.	Ambos observadores reflejan el cambio simultáneamente en tiempo real (ej. 'ACTUALIZADO: Angelica (5)' visible en el panel principal).

CONCLUSIONES

- Consideramos que la implementación de la arquitectura de tres capas fue fundamental para organizar adecuadamente el sistema, ya que permitió separar las responsabilidades entre las capas de Datos, Lógica de Negocio y Presentación, facilitando el mantenimiento y comprensión del proyecto.
- Observamos que el patrón Adapter resolvió de forma efectiva el problema de compatibilidad entre formatos de datos externos y el modelo interno del sistema. Gracias a este patrón fue posible integrar información proveniente de diferentes fuentes sin modificar la estructura principal del CRUD.
- Creemos que el patrón Decorator aportó una solución flexible para incorporar funcionalidades adicionales, como auditoría y validaciones, evitando cambios directos en las clases base y favoreciendo la extensibilidad del sistema.
- Comprobamos que el patrón Observer facilitó la gestión de eventos dentro de la aplicación, permitiendo que distintos componentes reaccionen automáticamente a las operaciones realizadas sobre los estudiantes sin generar dependencias innecesarias entre ellos.

RECOMENDACIONES

- Recomendamos mantener la separación de responsabilidades entre capas para conservar una arquitectura organizada y facilitar futuras modificaciones o ampliaciones del sistema.
- Sugerimos continuar utilizando el patrón Adapter para integrar nuevas fuentes de información externas, evitando cambios innecesarios en las clases internas del proyecto.
- Consideramos conveniente emplear Decorator para agregar futuras funcionalidades transversales, como generación de reportes, control de acceso o registro avanzado de actividades.

- *Recomendamos gestionar adecuadamente el registro y eliminación de observadores dentro del patrón Observer para evitar referencias innecesarias y garantizar un comportamiento eficiente del sistema.*